

complement Linux security rather than replace it. File permissions, containers, namespaces, seccomp, AppArmor, and SELinux can reduce what a process can do even if the agent-level control misses something. The two layers solve different problems.

Linux controls are strong at enforcing operating system boundaries. Agent-level policy can reason about higher-level intent before those boundaries are reached. It can distinguish a normal ‘git status’ from a force-push, a read from a write, or a package inspection from an installation that may run lifecycle scripts. A useful deployment has both: semantic checks close to the tool call and operating system limits underneath.

Treat tools as capabilities

A practical policy starts by treating every tool as a capability rather than giving an agent a single all-or-nothing trust level. Most agents need far fewer permissions than the human account that launches them.

For example, a coding agent may be allowed to read the project tree, write only inside the current repository, run a known test command, and inspect Git state. It may require review before installing a package, pushing to a remote, or making a network request. It should normally be denied access to SSH keys, cloud credentials, browser profiles, password stores, and unrelated home-directory files unless the task specifically requires them.

This capability model also makes policies easier to audit. Instead of asking whether an agent is ‘trusted’, you can answer concrete questions: which directories may it modify, which commands may it launch, which domains may it contact, and which operations require a human decision?

Keep policy deterministic

Security policy should be predictable. Avoid making the final allow-or-deny decision with another language model.

A model can help classify context or explain why a rule fired, but the enforcement decision should come from deterministic rules whenever possible.

Teams can implement those rules directly in application code, in a small configuration format, or with an open policy engine such as Open Policy Agent. What matters is that the same proposed action produces the same decision under the same policy. That makes failures reproducible and lets administrators review changes in version control.

Escalate ambiguous actions

Blocking every mutation makes an agent safe but not very useful. Allowing every mutation creates the opposite problem. The review state gives teams a middle path for actions that are legitimate but unusually risky.

Good review candidates include a force-push, a recursive delete outside the workspace, a command that reads from a credential directory, a package installation with lifecycle scripts, or a network request that carries environment-derived secrets. The goal is not to ask a human about every command. It is to reserve review for the small set of actions where the cost of a mistake is high, or the policy lacks enough context to decide safely.

Test policy like application code

A policy that is not tested will drift. Treat security rules as code and build a small regression suite around the actions your agents actually perform. Each test should describe a proposed action and the decision you expect.

Useful tests include allowing a read inside the repository, denying reads from ~/.ssh, allowing the project test runner, reviewing a new outbound network destination, and denying

a destructive command outside the workspace. Add a regression test whenever a real incident or near miss exposes an unexpected path. This turns operational lessons into durable controls rather than one-time warnings.

Where HOL Guard fits

HOL Guard on open source implementation of this pre-execution pattern for AI agents and coding-tool workflows. It intercepts supported actions before execution and applies policy so a risky operation can be allowed, denied, or stopped for review. The broader pattern is not specific to one product. Teams can build the same architecture with their own wrappers, agent hooks, policy middleware, and Linux controls.

The important design choice is where enforcement lives. If the check sits after the command, it is monitoring. If the model can bypass the check, it is advisory. If every side-effecting tool call must cross a policy boundary before execution, it becomes an actual control.

Build the smallest useful boundary

Start with the actions that have the largest blast radius: shell execution, filesystem writes, credential access, package installation, Git pushes, and outbound network calls. Normalise those actions, apply deterministic policy, and make review a first-class result rather than an exceptional path.

AI agents will keep getting more capable. The useful security question is not whether a model can be perfectly trusted. It is whether a proposed action can be checked before the system commits to it. On Linux, that boundary can be small, testable, open source, and layered with controls administrators already understand. **END** 🐧

By: Michael Kantor

The author is the president of Hashgraph Online. He works on open standards and open source security tooling for AI agents.